



UNIVERSIDADE PAULISTA

ICET - INSTITUTO DE CIÊNCIAS EXATAS E TECNOLOGIA

**CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E
DESENVOLVIMENTO DE SISTEMAS**

PROJETO INTEGRADO MULTIDISCIPLINAR

PIM II

**Desenvolvimento de um Sistema Acadêmico Colaborativo com
Apoio de IA**

SÃO JOSÉ DOS CAMPOS – SP

NOVEMBRO / 2025

	RA
ARTHUR FERNANDES REIS	H59DAH1
JOÃO EDUARDO SILVA NEIVA NEU	H658GF0
JOÃO PEDRO AMÉRICO DA SILVA	H70IFB0
LUCAS MATHEUS MARQUES GONÇALVES	F3618A6
MIGUEL DINIZ BARRETO DOS SANTOS FILHO	R873528
VITOR DE MOURA DA SILVA	H6619J8

Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA

Projeto Integrado Multidisciplinar (PIM) desenvolvido como exigência parcial dos requisitos obrigatórios à aprovação semestral no Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da UNIP (Universidade Paulista), orientado pelo corpo docente do curso.

São José dos Campos – SP

Novembro / 2025

RESUMO

O trabalho apresentou o desenvolvimento de um Sistema Acadêmico Colaborativo com apoio de Inteligência Artificial (IA), elaborado como parte do Projeto Integrado Multidisciplinar (PIM II) do curso de Análise e Desenvolvimento de Sistemas da Universidade Paulista. O projeto teve como objetivo criar uma plataforma digital capaz de otimizar o gerenciamento de turmas, alunos e atividades acadêmicas, substituindo processos manuais por soluções automatizadas e sustentáveis. Para atingir esse propósito, aplicaram-se metodologias de Engenharia de Software Ágil, com ênfase no Scrum, e técnicas de modelagem UML, que permitiram a elaboração de diagramas de caso de uso, classes e sequência, representando as interações entre os usuários e o sistema. O desenvolvimento envolveu o uso das linguagens C e Python, explorando estruturas de dados, integração entre módulos e interface gráfica via Tkinter, além da implementação de algoritmos de IA para análise de desempenho e geração de relatórios inteligentes. Os resultados indicaram um sistema funcional, seguro e adaptável, capaz de operar em rede local e atender diferentes perfis de usuários (aluno, professor, coordenador e administrador). Concluiu-se que a integração entre tecnologia, colaboração e sustentabilidade promoveu ganhos significativos na gestão acadêmica, contribuindo para a transformação digital e a eficiência das rotinas educacionais.

Palavras-chave: Sistema acadêmico; Engenharia de software; Inteligência artificial; Python; Sustentabilidade.

Sumário

1. INTRODUÇÃO	5
2. PROGRAMAÇÃO ESTRUTURADA EM C.....	8
3. ENGENHARIA DE SOFTWARE ÁGIL.....	11
4. ESTRUTURA DE DADOS EM PYTHON.....	13
5. ANÁLISE E PROJETO DE SISTEMAS	15
6. REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS	19
7. EDUCAÇÃO AMBIENTAL.....	21
8. INTELIGÊNCIA ARTIFICIAL	22
9. PESQUISA, TECNOLOGIA E INOVAÇÃO	24
10. DESENVOLVIMENTO DO PROJETO	26
11. CONCLUSÃO	50
12. REFERÊNCIAS	51

1. INTRODUÇÃO

O avanço da tecnologia tem transformado a forma como as instituições de ensino administram suas atividades. Diante disso, este Projeto Integrado Multidisciplinar (PIM II) propõe o desenvolvimento de um sistema acadêmico colaborativo com apoio de Inteligência Artificial com o objetivo de facilitar o gerenciamento de turmas, alunos e atividades em um ambiente integrado e acessível em rede local.

A proposta visa substituir métodos manuais e descentralizados, como planilhas e e-mails, por uma plataforma automatizada e sustentável, que traga benefícios como a redução no uso de papel e promova maior eficiência nas rotinas escolares. O sistema será desenvolvido com base em práticas de Engenharia de Software Ágil, utilizando a metodologia Scrum e Diagramas UML para o planejamento e modelagem das funcionalidades.

Já as linguagens de programação “C” e Python, serão aplicadas na implementação de módulos específicos, garantindo desempenho e usabilidade. Além disso, serão incorporados recursos de Inteligência Artificial para otimizar processos e apoiar a tomada de decisões. Dessa forma, o projeto busca integrar tecnologia, colaboração e sustentabilidade, contribuindo para o aprimoramento da gestão educacional.

TEMA

Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA

OBJETIVO GERAL

Projetar e implementar um sistema acadêmico integrado que permita gerenciar turmas, alunos, aulas e atividades, com funcionalidades de colaboração, explorando práticas de engenharia de software ágil e uso de recursos de Inteligência Artificial.

Objetivos Específicos

- Aplicar engenharia de software ágil para organizar sprints, backlog e acompanhamento do projeto.
- Implementar algoritmos e estruturas de dados em Python aplicados a funcionalidades de busca, ordenação e relatórios.
- Desenvolver módulos críticos em C estruturado para compreender a base de sistemas mais próximos do hardware.
- Criar modelos de análise e projeto de sistemas, incluindo diagramas UML.
- Aplicar conceitos de redes de computadores e sistemas distribuídos, garantindo que o sistema funcione em rede local, com usuários distintos acessando simultaneamente, e explorando conceitos de cliente-servidor.
- Usar a Inteligência Artificial no desenvolvimento.
- Realizar pesquisa de tecnologias emergentes e propor inovações aplicáveis ao sistema.
- Incluir recomendações de educação ambiental (ex.: relatórios digitais em substituição ao papel, métricas de sustentabilidade).

CONTEXTUALIZAÇÃO DO CASO

Uma instituição de ensino necessita de um sistema colaborativo para apoiar professores e alunos no gerenciamento de turmas, aulas e atividades. Atualmente, controles são realizados de forma descentralizada (planilhas, e-mails, mensagens em aplicativos).

O sistema deve permitir cadastro de turmas e alunos, registro de aulas e diário eletrônico, upload e consulta de atividades, e módulos distribuídos em uma rede. Um dos objetivos é a eliminação do uso de papel pelos professores como medidas sustentáveis.

2. PROGRAMAÇÃO ESTRUTURADA EM C

O Sistema Acadêmico, desenvolvido em linguagem C, tem como finalidade simular o funcionamento básico de um sistema utilizado por instituições de ensino para cadastrar, organizar e gerenciar informações de alunos, professores, coordenadores e administradores.

O objetivo é demonstrar, de maneira prática, como a linguagem C pode ser aplicada na gestão de dados e controle de acesso de usuários.

Estrutura inicial do sistema:

Ao iniciar o sistema, o usuário é apresentado a um menu principal com as seguintes opções:

A- Criar conta – onde é possível escolher o tipo de perfil (aluno, professor, coordenador ou administrador) e registrar um novo usuário com e-mail e senha;

B- Já tenho conta (login) – que permite ao usuário acessar o sistema informando seu e-mail e senha cadastrados;

C- Sair – para encerrar o programa.

D- Criação de contas e perfis.

Durante a criação de uma conta, o sistema pede que o usuário escolha o tipo de perfil que deseja registrar, pois cada tipo de usuário possui funções e permissões distintas.

Se o perfil escolhido for o de aluno, o sistema abre um formulário para o preenchimento de dados pessoais, como nome, idade e curso. Após fornecer essas informações, o sistema gera um ID numérico exclusivo para o aluno, que serve como identificador no banco de dados. Esse ID é salvo junto com os outros dados para futuras consultas ou registros de turmas e atividades.

Tratando-se do armazenamento dos dados, os dados informados pelos usuários são armazenados em arquivos de texto no formato CSV (Comma Separated

Values), funcionando como um banco de dados simplificado e que utiliza arquivos separados para cada categoria de informação conforme abaixo:

- **usuarios.csv** – armazena o tipo de usuário, e-mail de login e senha;
- **alunos.csv** – guarda os dados pessoais de cada aluno, como ID, nome, idade, curso e número da turma;
- **turmas.csv** – registra as turmas criadas pelo coordenador;
- **atividades.csv** – contém as atividades cadastradas pelos professores, relacionadas ao número da turma e ao professor responsável.

O sistema também possui menus específicos para cada tipo de usuário, acessados após o login, sendo que:

- **O aluno** pode visualizar as atividades de sua turma;
- **O professor** pode criar, listar ou excluir atividades;
- **O coordenador** tem acesso às funções administrativas, como cadastrar turmas e listar alunos;
- **O administrador** possui permissões gerais de controle e visualização de todos os dados.

Lógica e funcionamento interno: A lógica do programa é baseada em estruturas de repetição e decisão, permitindo uma navegação simples e contínua pelos menus. Cada ação executada (como cadastrar, listar ou excluir informações) é registrada nos arquivos CSV por meio de funções específicas que abrem, escrevem ou leem os dados e, em seguida, fecham os arquivos corretamente, evitando a perda de informações.

Características e objetivos do sistema: O sistema foi desenvolvido com foco em organização, clareza e portabilidade. Por utilizar arquivos de texto simples e a linguagem C, o programa pode ser executado na maioria dos computadores, sem a necessidade de softwares complexos ou bancos de dados externos, o que facilita sua adesão.

Além disso, sua estrutura modular facilita futuras melhorias, como a inclusão de novos tipos de cadastros ou funcionalidades adicionais.

Em resumo, o programa em linguagem C foi utilizado para criar um sistema funcional de gerenciamento acadêmico, capaz de manipular dados reais, controlar acessos e manter informações organizadas em arquivos. Seu propósito é educacional, voltado ao aprendizado de lógica de programação, manipulação de arquivos, estruturas de dados e controle de fluxo dentro de um contexto prático e próximo da realidade de um sistema de gestão escolar.

3. ENGENHARIA DE SOFTWARE ÁGIL

A Engenharia de Software é uma área essencial dentro do desenvolvimento de sistemas, pois oferece métodos e práticas que garantem a qualidade, a organização e a eficiência de todo o processo de criação de software. Neste projeto, essa disciplina foi aplicada desde o planejamento inicial até a documentação final, orientando as etapas de desenvolvimento de forma estruturada e colaborativa.

O objetivo desta seção é apresentar como os conceitos da Engenharia de Software, aliados à metodologia ágil Scrum, foram utilizados para conduzir o desenvolvimento do sistema, assegurando que o produto final atendesse às necessidades dos usuários, com qualidade, segurança e facilidade de manutenção.

A Engenharia de Software é responsável por planejar, organizar e garantir a qualidade no desenvolvimento de sistemas. Segundo Sommerville (2019), ela aplica princípios científicos e boas práticas de engenharia para criar softwares confiáveis, eficientes e dentro de prazos e custos definidos. No projeto, a Engenharia de Software orientou desde o início a construção e documentação do sistema, com o objetivo de criar uma ferramenta que integrasse alunos e professores em um ambiente digital funcional e acessível.

Além de estruturar o desenvolvimento, a Engenharia de Software é essencial para a documentação, registrando decisões, funcionalidades e processos implementados. Pressman e Maxim (2021) destacam que a documentação facilita o entendimento, a manutenção e a evolução do software, enquanto Winters, Manshreck e Wright (2020) reforçam sua importância para reduzir a dependência de conhecimento individual e aumentar a eficiência das equipes.

O desenvolvimento começou com o levantamento de requisitos, definindo necessidades dos usuários e funções do sistema, como cadastro de alunos, professores e turmas; registro de aulas e presenças; envio e consulta de atividades; geração de relatórios automáticos; comunicação entre usuários; e um módulo de Inteligência Artificial para análise de desempenho e apoio na correção de atividades. Também foram previstos autenticação e controle de acesso para diferentes perfis, com operação em rede local (LAN) para múltiplos usuários simultâneos.

Os requisitos não funcionais asseguram a qualidade do sistema, incluindo desempenho rápido, interface intuitiva, segurança de dados (conforme LGPD),

confiabilidade no armazenamento, compatibilidade com diferentes sistemas, sustentabilidade e facilidade de manutenção.

O projeto utilizou a metodologia ágil Scrum, que organiza o trabalho em sprints curtas e iterativas (Schwaber e Sutherland, 2020). Três papéis principais guiam o processo: Product Owner (prioriza funcionalidades), Scrum Master (garante a aplicação do Scrum) e equipe de desenvolvimento (implementa o sistema). As etapas incluem planejamento, reuniões diárias, revisão de entregas e retrospectiva para melhorias, essa abordagem permitiu divisão clara de tarefas, acompanhamento constante e ajustes rápidos.

A combinação da Engenharia de Software com Scrum resultou em um processo colaborativo, estruturado e alinhado aos objetivos iniciais. Como afirmam Pressman e Maxim (2021), a Engenharia de Software moderna equilibra rigor técnico e flexibilidade ágil, produzindo sistemas sustentáveis e capazes de evoluir continuamente.

4. ESTRUTURA DE DADOS EM PYTHON

4.1 Caracterização da Linguagem e Arquitetura de Dados

A escolha do Python, foi estratégica para o desenvolvimento da camada de *front-end* e controle do Sistema Acadêmico. A linguagem oferece uma sintaxe de alto nível que otimiza a produtividade, característica essencial para a metodologia ágil adotada (Capítulo 3).

O projeto adota uma arquitetura de dados baseada em persistência via arquivos CSV, utilizando as estruturas nativas do Python:

- **Dicionários (*dict*):** Cada registro de dados (aluno, nota, atividade) é tratado como um dicionário, onde as chaves correspondem aos *fieldnames* (cabecinhos do CSV) e os valores, aos dados do registro.
- **Listas (*list*):** Os arquivos CSV são lidos e processados como listas de dicionários, permitindo a manipulação e a filtragem de coleções de dados de forma eficiente.

4.2 Requisitos Técnicos de Implementação

O sistema implementa o uso de estruturas de dados e controle de fluxo do Python para atender diretamente aos requisitos funcionais e não-funcionais definidos.

4.2.1 Utilização de Estruturas de Decisão e Repetição

Estrutura	Função no Sistema	Exemplo de Aplicação
Decisão (if/elif/else)	Gerenciamento de Acesso e Validação de Dados.	O método do_login utiliza estruturas de decisão para determinar o perfil do usuário (Aluno, Professor, Coordenador, etc.) e redirecioná-lo para o <i>dashboard</i> correto, validando o tipo de acesso.
Repetição (for)	Processamento de Dados, Filtros e Relatórios.	A função ver_atividades itera sobre a lista completa de atividades (ARQ_ATIV) para filtrar e listar apenas aquelas que pertencem à turma do aluno logado e cujo ID não consta no registro de entregas (ARQ_ENTREGA).

A lógica de negócio do sistema é controlada por estas estruturas, garantindo a segurança e o processamento de grandes volumes de dados de forma escalável:

4.2.2 Interação com Outras Plataformas de Desenvolvimento (Linguagem C)

A estratégia de integração entre o módulo Python e os módulos de baixo nível (como o eventual módulo em C, conforme requisito) é estabelecida através da troca de dados via sistema de arquivos. O Python utiliza a biblioteca csv para garantir a leitura e a escrita padronizadas dos dados em arquivos (e.g., alunos.csv, notas.csv). Esta abordagem permite que qualquer outra plataforma (como um binário em C) possa acessar ou atualizar a base de dados do sistema, mantendo a interoperabilidade e garantindo que o módulo em C consiga fazer a leitura e salvar os dados em arquivo, conforme exigido.

4.2.3 Criação de Soluções Eficientes de Interface (tkinter)

A interface de usuário foi projetada para ser autoexplicativa e responsiva, utilizando o módulo tkinter do Python para construir um *Graphical User Interface* (GUI) nativo. A eficiência da solução se manifesta em:

- Modularidade de Telas: O método `clear()` e as funções `show_*` garantem que a tela seja completamente reconstruída ao trocar de perfil ou menu, otimizando o uso de recursos.

Sincronização Assíncrona (*Polling*): O método `poll(self)` utiliza a função `root.after()` para checar periodicamente o tempo de modificação (*timestamp*) dos arquivos CSV. Essa técnica garante que, se outro usuário (em outra máquina, simulando ambiente de rede) modificar a base de dados, a tela do usuário atual será recarregada automaticamente, promovendo a colaboração e a coesão dos dados em um ambiente distribuído.

5. ANÁLISE E PROJETO DE SISTEMAS

A etapa de Análise e Projeto de Sistemas representa uma das fases mais relevantes no desenvolvimento de software, pois define como o sistema proposto será estruturado, quais serão seus componentes e de que forma ocorrerá a interação entre os usuários e as funcionalidades.

No contexto do Sistema Acadêmico, esta etapa tem como objetivo detalhar a lógica de funcionamentos e os processos que sustentam o sistema, utilizando a Linguagem de Modelagem Unificada (UML). A UML permite representar graficamente a arquitetura do sistema por meio de diagramas que facilitam a compreensão dos fluxos de informação, a comunicação entre atores e a estrutura de classes e objetos. Assim, a modelagem garante maior clareza e padronização durante o desenvolvimento e manutenção do software.

5.1 Lógica e Metodologia Adotada

A modelagem do Sistema Acadêmico foi elaborada com base na metodologia UML, adotando o padrão de camadas arquiteturais do tipo Boundary, Control e Entity. Essa abordagem separa as responsabilidades do sistema em três níveis distintos:

- **Boundary (Interface):** representa as telas ou pontos de interação com o usuário (por exemplo, a tela de login)
- **Control (Controle):** responsável pelo processamento e validação e controle de fluxo entre interface e dados.
- **Entity(Entidade):** representa os dados que serão armazenados e manipulados pelo sistema., como usuários, alunos, professores e coordenadores.

A escolha da UML e da arquitetura em camadas foi motivada pela necessidade de organização, reutilização de código e clareza estrutural, garantido um desenvolvimento mais consistente e escalável

5.2 Documentação

5.2.1 Especificação de Requisitos

Tem como objetivo definir e descrever as funcionalidades esperadas do sistema, bem como as restrições e características de qualidade que devem ser observadas durante o seu desenvolvimento.

Os requisitos foram divididos em duas categorias: funcionais e não funcionais, exemplos:

RF01: Gerenciamento de turmas: O coordenador pode criar, editar ou excluir turmas, atribuir professores e alunos a cada uma delas, e supervisionar a organização acadêmica do sistema.

RF02: Acesso as atividades: Permite que o aluno visualize as notas lançadas pelos professores em suas disciplinas, acompanhando seu desempenho acadêmico.

RF03: Edição de Notas: O professor e o coordenador podem lançar, alterar ou corrigir as notas dos alunos em cada atividade, garantindo o controle e atualização do desempenho acadêmico.

RNF01: Desempenho: As consultas e operações do sistema devem ser capazes de serem executadas sem travaremos.

RNF02: Usabilidade: O sistema deve apresentar interface simples, intuitiva e acessível aos diferentes perfis de usuários.

RNF03: Confiabilidade: O sistema deve garantir integridade das informações durante o acesso.

5.3 Diagramas UML Desenvolvidos

5.3.1 Diagrama de Casos de Uso

O diagrama de casos de uso representa os atores e suas interações com o sistema. Foram identificados quatro principais perfis de usuários:

Aluno: Realiza login, acessa atividades e consulta notas.

Professor: Adiciona, edita e exclui atividades, além de editar notas.

Coordenador: Também adiciona, edita e exclui atividades e além de gerenciar turmas e registra alunos.

Administrator (ADM): Valida a senha master e gera relatórios administrativos.

Cada ator interage com casos de uso específicos, garantindo a divisão adequada de responsabilidade e acesso às funcionalidades de acordo com o perfil.

5.4.2 Diagrama de Classes

O diagrama de classes reflete a estrutura lógica do sistema e as relações entre os componentes. A classe usuário atua como base, sendo herdada por Aluno, Professor, Coordenador e ADM, cada qual com atributos e operações específicos.

Exemplo de classes modeladas:

Aluno: Contém métodos para acessarAtividades () e acessarNotas ().

Professor: Possui operações como: adicionarAtividades () e editarNotas ().

Coordenador: Responsável por gerenciarTurmas () e registrarAluno ().

ADM: Implementa validação SenhaMaster() e gerarRelatório ().

Além das classes principais, foram conectadas por meio de associações com multiplicidades adequadas (1..1, 1..*, etc.) estrutura essa que garante a facilidade da compreensão

5.4.3 Diagrama de Sequência

Os diagramas de sequência ilustram o fluxo temporal das interações entre os objetos, entre os casos representados, destacam-se:

- **Login e Validação de Credenciais:** Mostra o fluxo de autenticação entre as classes Login_, Cd_Login e Usuário.
- **Gerenciar Turmas:** Representa a sequência de mensagens trocadas entre o coordenador e as classes responsáveis pela criação e gerenciamento de turmas.

- **Editar Notas e Adicionar Atividades:** Detalham as interações entre o professor, o sistema e as classes de controle correspondentes, esses diagramas demonstram de forma visual a troca de mensagens e o encadeamento das ações do sistema.

5.5 Arquitetura e Padrões Utilizados

A proposta segue princípios de orientações a objetos, garantido que cada classe desempenhe apenas uma função específica, utilizamos o modelo de Boundary (View), Control(Controller) e Entity (Model), já aplicados nos diagramas desenvolvidos, esse padrão é amplamente utilizado em sistemas educacionais por permitir flexibilidade e manutenção simplificada.

5.6 Justificativas Técnicas

A utilização da UML como metodologia de modelagem foi escolhida por sua aceitação internacional, flexibilidade visual, compatibilidade com ferramentas de engenharia de software, como o Astah.

Além disso, a separação entre as classes Boundary e Control facilita futuras implementações e integrações com banco de dados, reforçando a consistência e manutenção do projeto.

6. REDES DE COMPUTADORES E SISTEMAS DISTRIBUÍDOS

A implementação de uma infraestrutura de rede de dados e comunicação tem como principal finalidade criar uma base tecnológica sólida que sustente o funcionamento eficiente dos sistemas digitais. O objetivo é garantir conectividade estável, comunicação segura entre os dispositivos e acesso ágil às informações, de forma a apoiar as atividades administrativas, operacionais e de ensino.

Os fundamentos das redes de dados se baseiam na interconexão de diferentes dispositivos como; Computadores, servidores, roteadores, switches e pontos de acesso que trocam informações por meio de cabos ou sinais sem fio. Essa troca de dados segue protocolos de comunicação padronizados, como o TCP/IP, responsáveis por assegurar a integridade e o encaminhamento correto das informações dentro da rede.

No contexto da infraestrutura proposta, a comunicação é organizada em camadas, de acordo com o modelo OSI (Open Systems Interconnection), que define as funções de cada etapa da transmissão de dados desde a camada física, responsável pelo meio de transmissão, até a camada de aplicação, onde os softwares interagem com o usuário final. Essa divisão em camadas facilita o gerenciamento da rede e o diagnóstico de falhas, além de permitir maior compatibilidade entre diferentes equipamentos e fabricantes.

A escolha dos equipamentos de rede é um ponto crítico para garantir o desempenho e a segurança. Roteadores são utilizados para controlar o tráfego e interligar redes distintas; switches permitem a comunicação eficiente entre dispositivos locais; servidores centralizam o armazenamento de dados e executam serviços de rede, como DHCP, DNS e autenticação de usuários; enquanto os pontos de acesso (Access Points) ampliam a conectividade sem fio, garantindo o acesso de múltiplos dispositivos à rede interna e à internet.

Além dos componentes físicos, é fundamental considerar a topologia da rede ou seja, a forma como os dispositivos estão interconectados. A topologia em estrela é a mais utilizada por proporcionar maior controle, facilidade de manutenção e redução de falhas. O desempenho do software utilizado nas máquinas está diretamente ligado à qualidade da rede, já que atrasos, perda de pacotes e instabilidade de conexão podem comprometer aplicações que dependem de comunicação contínua com servidores ou sistemas em nuvem.

Dessa forma, o projeto de rede deve contemplar não apenas a instalação dos equipamentos, mas também práticas de cibersegurança e manutenção preventiva. Isso inclui o controle de acesso, atualizações periódicas, criação de senhas seguras, segmentação de rede (VLANs) e uso de protocolos de criptografia, garantindo a integridade e confidencialidade das informações trafegadas.

7. EDUCAÇÃO AMBIENTAL

A Educação Ambiental é um dos pilares fundamentais para a construção de uma sociedade mais consciente, responsável e sustentável. Seu objetivo é promover a compreensão sobre a interdependência entre o ser humano e o meio ambiente, estimulando mudanças de comportamento e atitudes que reduzam os impactos negativos sobre os ecossistemas. Segundo Dias (2017), a Educação Ambiental deve ser entendida como um processo contínuo de aprendizado, que envolve o desenvolvimento de valores, conhecimentos e práticas voltadas à preservação e melhoria da qualidade de vida.

No contexto tecnológico, a Educação Ambiental assume papel estratégico, pois os sistemas computacionais têm potencial para disseminar informações e práticas sustentáveis em larga escala. Leff (2011) ressalta que a integração entre tecnologia e sustentabilidade é essencial para promover uma nova racionalidade ecológica, na qual o avanço tecnológico se desenvolve de forma harmônica com o equilíbrio ambiental. Essa perspectiva reforça a necessidade de adotar ferramentas digitais como aliadas na conscientização e formação de comunidades mais responsáveis com o meio ambiente.

Neste projeto, a disciplina de Educação Ambiental busca demonstrar como os recursos tecnológicos podem ser utilizados para reduzir impactos ambientais, promover o uso eficiente da energia e estimular a adoção de boas práticas ecológicas.

Também se destaca o papel dos sistemas computacionais na divulgação de informações ambientais — permitindo que usuários, empresas e instituições adotem medidas conscientes e sustentáveis. Assim, a disciplina contribui não apenas para a formação técnica, mas também para o desenvolvimento ético e socioambiental de todos os envolvidos.

8. INTELIGÊNCIA ARTIFICIAL

A Inteligência Artificial (IA) é uma área da ciência da computação que se destaca por permitir o desenvolvimento de sistemas capazes de aprender, raciocinar e tomar decisões com base em dados. Conforme apontam Russell e Norvig (2010), a IA busca compreender e reproduzir o comportamento humano em máquinas, possibilitando que sistemas computacionais realizem tarefas complexas e aprimorem seu desempenho com o tempo.

No contexto tecnológico atual, a IA tem sido amplamente aplicada para transformar grandes volumes de dados em conhecimento útil, especialmente no campo educacional. Na educação digital, a IA tem proporcionado avanços significativos ao tornar os sistemas mais inteligentes, interativos e personalizados. Holmes, Bialik e Fadda (2019) destacam que a IA favorece a personalização do ensino, ajustando conteúdos e metodologias às necessidades de cada aluno, o que aumenta o engajamento e a eficiência no aprendizado.

Além disso, a IA contribui na gestão administrativa, automatizando tarefas e fornecendo relatórios precisos que auxiliam professores e gestores. Mitchell (1997) reforça que os algoritmos de aprendizado de máquina permitem identificar padrões complexos e apoiar a tomada de decisão com base em dados confiáveis.

No sistema proposto neste projeto, a aplicação da IA foi estruturada em uma arquitetura modular voltada ao tratamento e análise de dados acadêmicos. O processo inicia-se com a coleta e pré-processamento de informações sobre notas, frequência e comportamento dos usuários, etapa essencial para garantir a qualidade dos dados e a precisão das análises (Goodfellow, Bengio e Courville, 2016).

Em seguida, modelos de aprendizado supervisionado são utilizados para identificar padrões de desempenho e prever dificuldades, gerando recomendações automáticas que apoiam a gestão e o acompanhamento educacional.

A arquitetura foi projetada para operar de forma contínua e escalável, aprimorando os algoritmos conforme novos dados são inseridos. Segundo Pressman e Maxim (2021), a integração entre engenharia de software e algoritmos inteligentes é fundamental para garantir estabilidade, desempenho e segurança em sistemas complexos. Assim, a IA não apenas otimiza processos, mas também transforma a

maneira como o conhecimento é produzido e distribuído, consolidando-se como elemento central da educação digital moderna.

Conforme destacam Holmes, Bialik e Fadda (2019), o verdadeiro valor da IA na educação está em sua capacidade de transformar dados em conhecimento prático, favorecendo decisões baseadas em evidências. Dessa forma, a Inteligência Artificial se estabelece como uma aliada essencial na criação de ambientes de aprendizagem mais inteligentes, dinâmicos e eficientes, alinhados às demandas da transformação digital no ensino.

9. PESQUISA, TECNOLOGIA E INOVAÇÃO

A Educação Ambiental, a Pesquisa, a Tecnologia e a Inovação são pilares fundamentais para o desenvolvimento sustentável e o avanço da sociedade contemporânea. A Educação Ambiental busca promover a conscientização e a responsabilidade socioambiental, incentivando atitudes que reduzam os impactos negativos sobre o meio ambiente. Segundo Dias (2017), trata-se de um processo contínuo de aprendizado que envolve a formação de valores e práticas voltadas à preservação da vida e à melhoria da qualidade ambiental. Quando aliada à tecnologia, essa abordagem ganha ainda mais força, pois os sistemas computacionais se tornam ferramentas estratégicas na disseminação de informações e na adoção de práticas sustentáveis em larga escala.

Leff (2011) reforça que a integração entre tecnologia e sustentabilidade é essencial para estabelecer uma nova racionalidade ecológica, em que o avanço tecnológico ocorre de forma equilibrada e responsável. Nesse sentido, os recursos digitais podem ser utilizados para promover o uso eficiente da energia, reduzir o desperdício e fomentar a conscientização ambiental. O projeto aqui desenvolvido reflete essa integração, utilizando soluções tecnológicas para incentivar a sustentabilidade digital e a adoção de práticas ecológicas no ambiente computacional.

A aplicação dos conceitos de Educação Ambiental dentro do sistema proposto prioriza ações como o uso racional de energia por meio de servidores eficientes, desligamento automático de equipamentos ociosos e substituição de materiais impressos por conteúdos digitais, o que diminui o uso de papel e amplia o acesso à informação. Conforme Philippi Jr. e Pelicioni (2005), pequenas iniciativas tecnológicas, quando aplicadas de forma contínua, podem gerar grandes impactos ambientais positivos.

Ao mesmo tempo, a pesquisa, a tecnologia e a inovação têm papel central no desenvolvimento de sistemas computacionais modernos e sustentáveis. Tidd e Bessant (2015) afirmam que a inovação vai além da criação de algo novo: ela consiste em aplicar o conhecimento de maneira prática para resolver problemas reais e agregar valor à sociedade. A pesquisa científica, nesse contexto, fornece as bases

metodológicas e conceituais que orientam o desenvolvimento de novas soluções digitais.

Em um cenário de transformação digital e globalização, as organizações dependem cada vez mais de tecnologias emergentes, como inteligência artificial, computação em nuvem e análise de dados, para manter-se competitivas e eficientes. Drucker (2014) destaca que a inovação tecnológica é o principal diferencial entre instituições que evoluem e aquelas que permanecem estagnadas.

De acordo com o Manual de Oslo (OECD, 2018), a inovação em tecnologia da informação deve estar associada à criação de conhecimento e à sua aplicação prática, envolvendo etapas de experimentação, prototipagem e avaliação contínua. Dessa forma, a disciplina de Pesquisa, Tecnologia e Inovação conecta teoria e prática, promovendo uma visão estratégica do desenvolvimento de sistemas informatizados, sempre orientada à evolução técnica, à sustentabilidade e à relevância social.

Ao unir Educação Ambiental, Pesquisa, Tecnologia e Inovação, o projeto propõe uma abordagem integrada que valoriza tanto o avanço tecnológico quanto a responsabilidade ecológica, contribuindo para a criação de sistemas mais eficientes, éticos e sustentáveis, capazes de equilibrar o progresso humano e a preservação ambiental.

10. DESENVOLVIMENTO DO PROJETO

10.1.1 Programação Estruturada Em C.

O presente documento tem como objetivo apresentar e explicar trechos essenciais do código-fonte desenvolvido em linguagem C para o Sistema Acadêmico, um projeto que simula o funcionamento básico de um sistema de gerenciamento escolar.

Por meio desta análise, busca-se demonstrar o funcionamento interno de partes fundamentais do programa, esclarecendo como ocorre o cadastro de usuários, o processo de autenticação (login) e a organização dos dados em memória e em arquivos CSV.

O sistema foi desenvolvido com o propósito de permitir que diferentes tipos de usuários — alunos, professores, coordenadores e administradores — possam criar contas, fazer login e interagir com o ambiente de forma simulada.

Cada ação realizada no programa é traduzida em operações lógicas e estruturadas, que manipulam dados gravados em memória e, posteriormente, armazenam essas informações em arquivos de texto no formato CSV, garantindo persistência entre execuções.

Os trechos de código apresentados a seguir foram cuidadosamente selecionados por representarem partes cruciais do sistema, responsáveis pela criação de usuários e pelo processo de autenticação. Cada imagem contém o respectivo trecho do código-fonte e, logo abaixo, uma explicação detalhada sobre sua função, lógica e importância dentro do programa.

Essa análise busca facilitar a compreensão da estrutura e do raciocínio lógico utilizados na implementação, servindo tanto como um guia de estudo para iniciantes na linguagem C quanto como um material de apoio para a apresentação do projeto.

Figura 1: Parte programa C.

```
220 void usuarios_criar_fluxo_escolha(void){
221     if(Nusuarios>=MAX_USERS){ puts("Limite de usuarios."); return; }
222     Usuario u; u.tipo[0]=u.login[0]=u.senha[0]='\0';
223
224     // 1) escolhe perfil
225     escolher_tipo_usuario(u.tipo,sizeof(u.tipo)); if(!u.tipo[0]) return;
226
227     // 2) cria email/senha com validação
228     while(1){
229         ler_linha("Digite o email: ", u.login, sizeof(u.login));
230         if(!email_valido(u.login)) { puts("Email invalido."); continue; }
231         if(existe_login(u.login)) { puts("Email ja cadastrado."); continue; }
232         break;
233     }
234     ler_linha("Crie uma senha: ", u.senha, sizeof(u.senha));
235
236     // 3) salva usuario
237     Nusuarios[Nusuarios++]=u; salvar_usuarios();
238     printf("\nConta criada com sucesso (%s)!\n", u.tipo);
239
240     // 4) se for ALUNO, já cadastra o aluno e mostra o ID
241     if(!strcmp(u.tipo,"aluno")){
242         alunos_cadastrar_via_login(u.login);
243     }
244 }
```

Fonte: O grupo, 2025.

Função: usuarios_criar_fluxo_escolha(void).

Essa função é chamada quando o usuário escolhe a opção “1) Criar nova conta” no menu inicial do programa.

Ela cuida de todo o processo de cadastro, desde a escolha do perfil até o salvamento no arquivo CSV.

Figura 2: Parte programa C.

```
if(Nusuarios>=MAX_USERS){ puts("Limite de usuarios."); return; }
Usuario u; u.tipo[0]=u.login[0]=u.senha[0]='\0';
```

Fonte: O grupo, 2025.

Aqui o programa verifica se o número máximo de usuários (MAX_USERS) já foi atingido. Se estiver cheio, mostra uma mensagem e interrompe o cadastro.

Cria uma estrutura do tipo Usuario criada no começo do código como uma variável global utilizada como um meio de banco de dados temporário. Inicializa as strings tipo, login e senha com caractere nulo ('0'), ou seja, começam vazias.

Figura 3: Parte programa C.

```
// 1) escolhe perfil  
escolher_tipo_usuario(u.tipo, sizeof(u.tipo)); if(!u.tipo[0]) return;
```

Fonte: O grupo, 2025.

- Chama a função `escolher_tipo_usuario()`, que pergunta qual perfil o usuário quer criar: **aluno, professor, coordenador ou administrador**.
- Se o usuário não escolher nada (string vazia), o cadastro é cancelado.

Figura 4: Parte programa C.

```
// 2) cria email/senha com validação  
while(1){  
    ler_linha("Digite o email: ", u.login, sizeof(u.login));  
    if(!email_valido(u.login)) { puts("Email invalido."); continue; }  
    if(existe_login(u.login)) { puts("Email ja cadastrado."); continue; }  
    break;  
}  
ler_linha("Crie uma senha: ", u.senha, sizeof(u.senha));
```

Fonte: O grupo, 2025.

O sistema entra em um loop (while) até o e-mail digitado ser considerado válido.

Primeiro verifica se o e-mail é válido (`email_valido()`).

Depois verifica se ele já existe no sistema (`existe_login()`).

Só sai do loop quando o e-mail for aceito.

Em seguida, pede para o usuário criar uma senha.

Figura 5: Parte programa C

```
// 3) salva usuario  
Vusuarios[Nusuarios++] = u; salvar_usuarios();  
printf("\nConta criada com sucesso (%s)!\n", u.tipo);
```

Fonte: O grupo, 2025.

O usuário recém-criado é adicionado ao vetor global Vusuarios.

Nusuarios++ aumenta o contador total de usuários.

A função salvar_usuarios() grava tudo no arquivo usuarios.csv.

Por fim, imprime uma mensagem de sucesso mostrando o tipo de conta criada.

Figura 6: Parte programa C.

```
// 4) se for ALUNO, já cadastra o aluno e mostra o ID
if(!strcmp(u.tipo,"aluno")){
    alunos_cadastrar_via_login(u.login);
}
```

Fonte: O grupo, 2025.

Se o tipo de usuário for “aluno”, o programa chama outra função: alunos_cadastrar_via_login(u.login). Essa função faz o cadastro completo do aluno, pedindo nome, idade, curso etc. Ela também gera um ID exclusivo para o aluno e mostra na tela.

Figura 7: Parte programa C.

```
Sessao s={0, "", ""};
if(Nusuarios==0){ puts("Nenhum usuario cadastrado. Use a opcao 1."); PAUSE(); return s; }
```

Fonte: O grupo, 2025.

Cria uma variável s do tipo Sessão, que representa o usuário logado e inicializa como “vazia” (autenticado = 0, login = ""). Isso serve para garantir que se o login falhar, a função devolva uma sessão inválida.

Se o sistema ainda não tem nenhum usuário (Nusuarios == 0), ele mostra a mensagem: “Nenhum usuário cadastrado. Use a opção 1.” evitando ao tentar fazer login sem dados no CSV. Depois, retorna s (a sessão vazia), encerrando a função.

Figura 8: Parte programa C.

```
char tipo[16]="";  
puts("\n=== LOGIN ===");  
escolher_tipo_usuario(tipo,sizeof(tipo)); if(!tipo[0]) return s;
```

Fonte: O grupo, 2025.

O programa pede que o usuário selecione seu perfil (Aluno, Professor, Coordenador ou Administrador). A função `escolher_tipo_usuario()` salva essa escolha dentro da variável `tipo`.

Caso o usuário não escolha nada (`!tipo[0] → string vazia`), a função retorna imediatamente, cancelando o login.

Figura 9: Parte programa C.

```
char e[MAX_STR], p[MAX_STR];  
ler_linha("Email: ", e, sizeof(e));  
ler_linha("Senha: ", p, sizeof(p));
```

Fonte: O grupo, 2025. 1

Aqui o programa pede o e-mail e a senha do usuário. Ambas são salvas nas variáveis `e` e `p`.

`ler_linha ()` é uma função auxiliar que lê a linha digitada e remove o `\n` no final.

Figura 10: Parte programa C

```
for(int i=0;i<Nusuarios;i++){  
    if(!strcmp(Vusuarios[i].tipo,tipo) && !strcmp(Vusuarios[i].login,e) && !strcmp(Vusuarios[i].senha,p)){  
        s.autenticado=1; strncpy(s.tipo,tipo,sizeof(s.tipo)); strncpy(s.login,e,sizeof(s.login));  
        return s;  
    }  
}
```

Fonte: O grupo, 2025.

O programa percorre o vetor global `Vusuarios[]`, que contém todos os usuários cadastrados. Para cada registro, ele compara três informações:

Tipo (aluno, prof, coord, adm);

E-mail (login);

Senha.

Se tudo for igual, o login é aceito:

s.autenticado = 1 marca a sessão como válida.

s.tipo e **s.login** armazenam os dados do usuário autenticado.

return s; encerra a função e devolve a sessão válida.

Figura 11: Parte programa C.

```
puts("Credenciais invalidas ou perfil incorreto."); PAUSE();  
return s;
```

Fonte: O grupo, 2025.

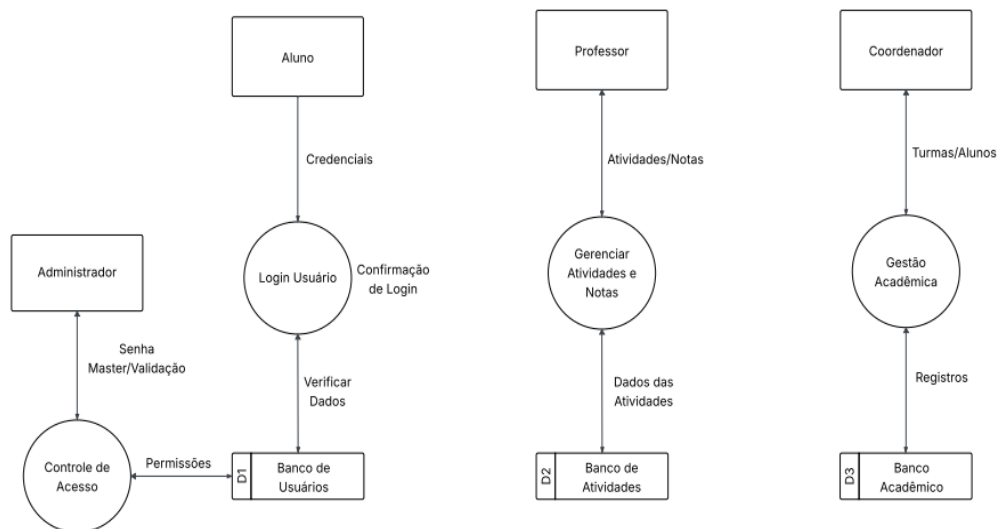
Se o loop terminou sem encontrar um usuário válido, mostra a mensagem de erro, retorna a sessão “vazia” (s.autenticado = 0), indicando que o login falhou.

Link do Repositório Git: <https://github.com/VitorMoura-coder/PIM>

10.2.1 Engenharia de Software Ágil.

O diagrama de fluxo de dados representa o funcionamento de um sistema acadêmico colaborativo, evidenciando a interação entre aluno, professor, coordenador e administrador. As três entidades externas no topo do diagrama realizam o login e acessam informações validadas, o professor gerencia atividades e notas, o coordenador administra turmas e alunos, e o administrador controla acessos e permissões. Esses processos integrados asseguram a organização, segurança e eficiência na gestão das informações acadêmicas.

Figura 12: Diagrama de Fluxo de Dados



Fonte: O grupo, 2025.

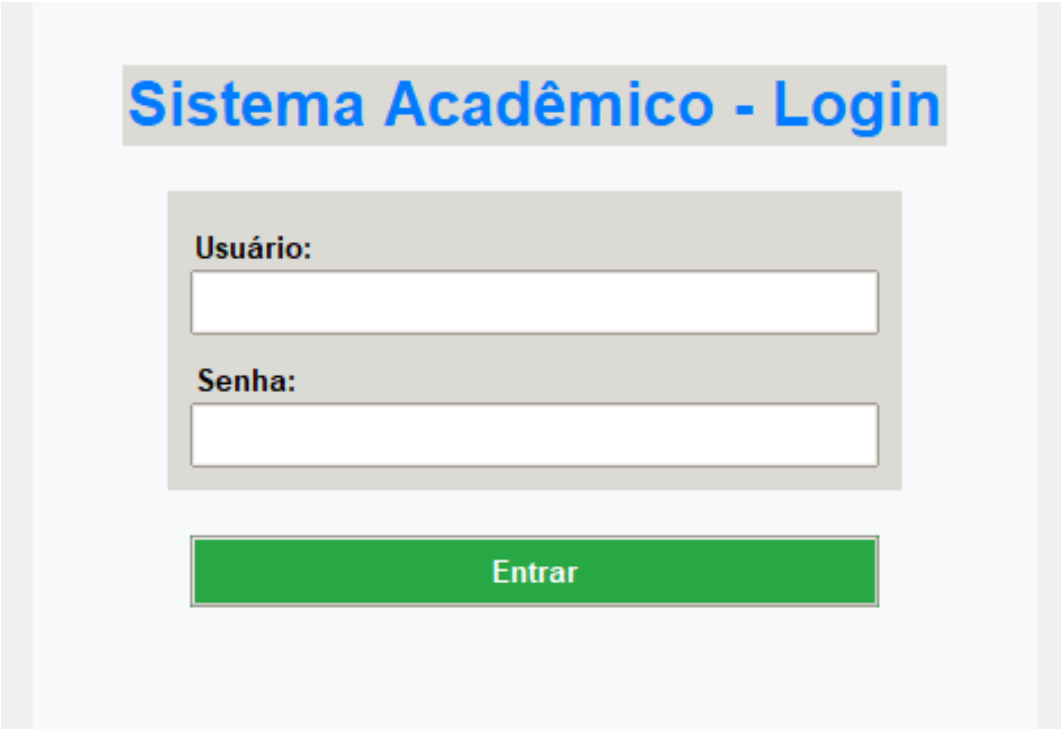
10.3.1 Desenvolvimento (Python - Estrutura de Dados e Interface)

A implementação do sistema foi realizada em Python e a persistência de dados em arquivos CSV, conforme a arquitetura de dados descrita no Capítulo 4. A seguir, são apresentados os *layouts* e os trechos de código que demonstram o uso prático dos requisitos de Estrutura de Dados em Python.

10.3.2 Soluções Eficientes de Interface (tkinter e Usabilidade)

O projeto utilizou a biblioteca tkinter para construir uma interface gráfica intuitiva, garantindo uma experiência de usuário eficiente. O Layout Principal de Login (Figura 13) foi projetado para ser limpo e direto, sendo o ponto de entrada para a lógica de decisão por perfil (item 4.2.1).

Figura 13: Tela de login do Sistema



A interface de login do Sistema Acadêmico apresenta um layout limpo e funcional. No topo, um cabeçalho azul contém o título "Sistema Acadêmico - Login" em letras brancas e em negrito. Abaixo, um formulário cinza claro contém dois campos de entrada: "Usuário:" e "Senha:", cada um seguido por um retângulo branco para digitação. No final do formulário, um botão verde com o texto "Entrar" em branco permite o acesso ao sistema.

Fonte: O grupo, 2025.

O Dashboard do Aluno (Figura 14) comprova a eficiência da interface ao apresentar botões focados em tarefas específicas (Consultar Cadastro, Ver Notas), que acessam e manipulam as estruturas de dados de forma modularizada.

Figura 14: Dashboard do Perfil do Aluno



Fonte: O grupo, 2025.

10.3.3 Aplicação de Estruturas de Decisão e Repetição

O código-fonte funcional utiliza Estruturas de Decisão para controle de acesso e Estruturas de Repetição para processamento e filtragem de dados.

A. Estrutura de Decisão: Controle de Acesso por Perfil

O método `do_login` demonstra o uso do `if/elif` para validar credenciais e, em seguida, direcionar o usuário para o *dashboard* específico de seu perfil, garantindo a separação de responsabilidades (Capítulo 5) e a segurança do acesso.

Figura 15: Método do Login

```
# Trecho de Código 10.2.1: Estrutura de Decisão para Login
# -----
def do_login(self):
    """Verifica credenciais e redireciona para a tela do perfil."""
    u,s=self.e_user.get().strip(),self.e_pass.get().strip()
    for row in ler_csv(ARQ_USU):
        if row.get("login")==u and row.get("senha")==s: # Decisão 1: Credenciais v
            self.usuario=row; typ=row.get("tipo")
            # Decisão 2: Redirecionamento por tipo de usuário (perfil)
            if typ.startswith("aluno"): self.show_aluno(u)
            elif typ.startswith("prof"): self.show_prof(u)
            elif typ.startswith("coord"): self.show_coord(u)
            elif typ.startswith("adm") or typ.startswith("tecnico"): self.show_adm
            return
    messagebox.showerror("Erro","Usuário ou senha incorretos.")
```

Fonte: O grupo, 2025.

B. Estrutura de Repetição: Filtro e Listagem de Atividades

A função `ver_atividades` utiliza o laço de repetição (`for`) para iterar sobre a lista de atividades (`ler_csv(ARQ_ATIV)`) e aplica uma condicional (decisão) interna para filtrar e exibir apenas as atividades pendentes da turma do aluno.

Figura 16: Estrutura de repetição

```
# Trecho de Código 10.2.2: Estrutura de Repetição para Filtragem de Dados
# -----
def ver_atividades(self, aluno):
    """Exibe atividades pendentes filtradas por turma e entregas."""
    turma_id, aluno_id = aluno.get("id_turma"), aluno.get("id")
    entregues_ids = get_entregues_ids(aluno_id) # IDs de atividades já entregues

    # Repetição: Filtra todas as atividades cadastradas
    ativs = [a for a in ler_csv(ARQ_ATIV)
             # Decisão: Condições de filtro para 'pendente'
             if a.get("id_turma") == turma_id
             and a.get("id_atividade") not in entregues_ids]

    if not ativs: # Decisão: Se a lista estiver vazia
        ttk.Label(main_frame, text="Nenhuma atividade pendente.").pack()
    else:
        # Repetição: Itera sobre a lista filtrada para popular a Listbox
        for a in ativs:
            box.insert(tk.END, f"ID: {a.get('id_atividade')} | Disciplina: ...")
```

Fonte: O grupo, 2025.

10.3.4 Estratégias de Interação de Dados e Módulos

O sistema utiliza funções modulares de CRUD (Create, Read, Update, Delete) para manipular a persistência em CSV, facilitando a interação com outras plataformas.

A. Leitura, Escrita e Persistência de Dados

O código a seguir demonstra o uso da função `ler_csv` (Leitura) e `escrever_csv` (Escrita/Atualização), que são a base da arquitetura de dados e permitiriam a integração com o módulo C através do arquivo compartilhado.

Figura 17: Interação e Persistência via CSV

```
# Trecho de Código 10.2.3: Interação e Persistência via CSV (Exemplo: Exclusão)
# -----
def excluir_atividade(self, box):
    """Remove a atividade selecionada e reescreve o arquivo CSV."""
    sel = box.curselection()
    if not sel: messagebox.showwarning("Aviso", "Selecione uma atividade.") ; return

    # Lógica para obter o ID da atividade selecionada (estrutura de string)
    aid = box.get(sel[0]).split("ID: ")[1].split(" |")[0].strip()

    # Leitura e Repetição: Carrega todos os registros
    ativs = ler_csv(ARQ_ATIV)

    # Decisão e Filtragem: Cria uma nova lista sem o registro a ser excluído
    ativs = [a for a in ativs if a.get("id_atividade") != aid]

    # Escrita: Sobrescreve o CSV inteiro com a lista filtrada (simula DELETE/UPDATE)
    escrever_csv(ARQ_ATIV, ATIV_FIELDS, ativs)
    messagebox.showinfo("Ok", "Atividade excluída.")
```

Fonte: O grupo, 2025.

O código-fonte completo e comentado que demonstra a aplicação prática de todas as estruturas e requisitos está disponível publicamente:

Link do Repositório Git: <https://github.com/VitorMoura-coder/PIM>

10.4.1 Análise e Projeto de Sistemas.

O **diagrama de casos de uso** representa as principais interações entre os atores e o sistema acadêmico colaborativo desenvolvido. Ele demonstra de forma

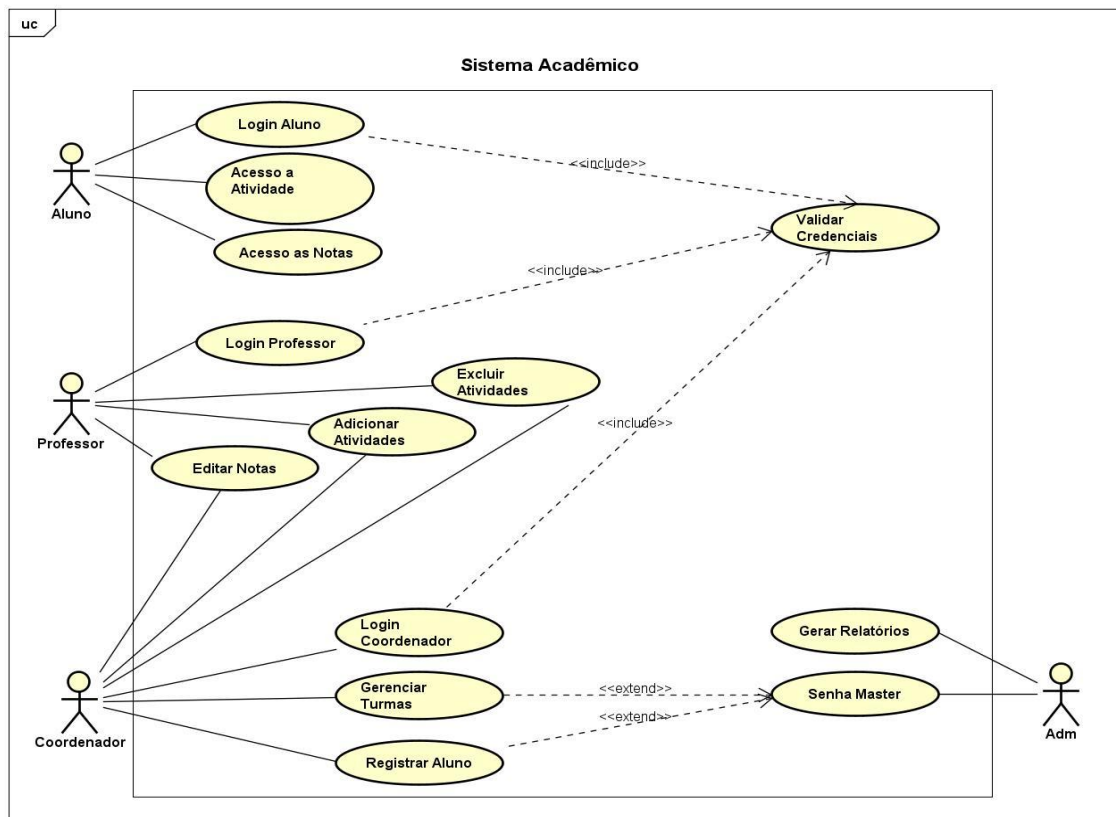
visual como cada tipo de usuário acessa as funcionalidades do sistema, evidenciando os relacionamentos e dependências entre os casos de uso.

No diagrama, observam-se quatro atores principais: Aluno, Professor, Coordenador e Administrador (Adm).

- O **Aluno** pode realizar o *Login Aluno*, *Acessar Atividade* e *Acessar as Notas*. Todas essas ações dependem do caso de uso *Validar Credenciais*, que foi representado com a relação **<<include>>**, indicando que a validação é obrigatória para o acesso ao sistema.
- O **Professor** possui casos de uso relacionados à gestão acadêmica, como *Login Professor*, *Adicionar Atividades*, *Excluir Atividades* e *Editar Notas*. Assim como o aluno, o login do professor também está vinculado ao caso *Validar Credenciais* por meio do relacionamento **<<include>>**, garantindo a autenticação antes de qualquer operação.
- O **Coordenador** tem acesso a funcionalidades administrativas, como *Login Coordenador*, *Gerenciar Turmas* e *Registrar Aluno*, também incluindo a validação de credenciais. Além disso, ele pode acessar o recurso de *Senha Master*, representado pela relação **<<extend>>**, que é ativada apenas em situações específicas de controle e segurança.
- O **Administrador (Adm)**, por sua vez, tem a responsabilidade de *Gerar Relatórios* e também utiliza a *Senha Master*, que se estende a partir de suas ações, demonstrando um nível de acesso privilegiado no sistema.

Esse diagrama permite visualizar de forma estruturada como as funções do sistema se organizam conforme o papel de cada usuário, evidenciando a importância da autenticação (*Validar Credenciais*) e do controle hierárquico de acessos (*Senha Master*). Dessa maneira, o modelo contribui para o entendimento do fluxo de interações e para a documentação do sistema dentro da metodologia UML, essencial no processo de Análise e Projeto de Sistemas.

Figura 18: Descrição do Diagrama de Casos de Uso – Sistema Acadêmico



Fonte: O grupo, 2025.

O **diagrama de classes** representa a estrutura lógica do Sistema Acadêmico Colaborativo, detalhando as principais classes, seus atributos, métodos e relacionamentos. Ele tem como objetivo demonstrar como os objetos do sistema se organizam e interagem para compor as funcionalidades descritas nos casos de uso.

No centro do modelo encontra-se a classe Usuário, que atua como classe genérica (superclasse) para as demais — *Aluno*, *Professor*, *Coordenador* e *Adm*. Essa herança evidencia que todos os tipos de usuários compartilham atributos e comportamentos comuns, como *id*, *nome*, *email*, *senha*, além das operações *login()* e *logout()*, garantindo padronização e segurança no processo de autenticação.

A classe Aluno possui atributos e métodos específicos, como *ra* e as operações *acessarAtividade()* e *acessarNota()*, conectando-se às classes Acesso Atividade e Acesso Nota, que armazenam dados sobre tarefas, notas e prazos. Essas classes refletem as funcionalidades acessadas pelo estudante dentro do sistema.

O Professor herda de *Usuário* e contém métodos voltados à gestão de conteúdo, como *adicionarAtividade()*, *excluirAtividade()* e *editarNota()*, além de se relacionar com classes auxiliares como Adicionar Atividade, Excluir Atividade e Editar Nota, responsáveis por detalhar os processos de criação, atualização e remoção de atividades acadêmicas.

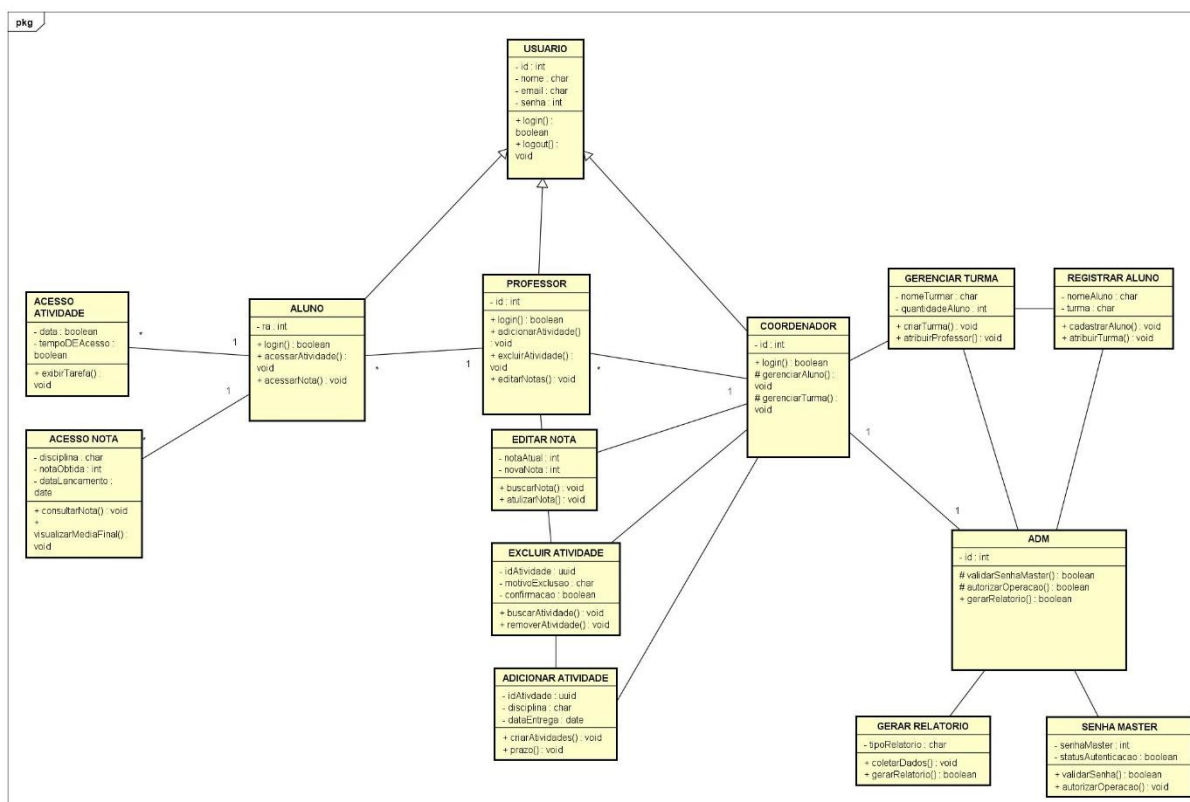
O Coordenador, também derivado da superclasse *Usuário*, possui métodos de administração interna, como *gerenciarAluno()* e *gerenciarTurma()*, além de manter relação com as classes Gerenciar Turma e Registrar Aluno, que definem operações de cadastro e atribuição de professores e turmas.

Já o Administrador (Adm) representa o nível mais alto de controle do sistema. Ele possui funções voltadas à segurança e auditoria, como *validarSenhaMaster()*, *autorizarOperacao()* e *gerarRelatorio()*. As classes Gerar Relatório e Senha Master complementam suas responsabilidades, permitindo que apenas usuários com credenciais especiais possam autorizar operações críticas e gerar relatórios gerenciais.

O diagrama também apresenta relacionamentos de multiplicidade, indicando que um coordenador pode gerenciar várias turmas e alunos, um professor pode criar diversas atividades, e um aluno pode acessar várias notas e tarefas. Esses relacionamentos refletem a estrutura real do ambiente acadêmico, garantindo consistência entre a modelagem e a lógica do sistema.

Em resumo, o diagrama de classes evidencia uma arquitetura organizada, modular e orientada a objetos, permitindo a reutilização de código, a segurança na autenticação e a clareza nas responsabilidades de cada componente. Ele serve como base para a implementação do sistema e assegura que o desenvolvimento siga os princípios da Engenharia de Software Ágil e da metodologia UML.

Figura 19: Descrição do Diagrama de Classes – Sistema Acadêmico



Fonte: O grupo, 2025.

10.4.2 Descrição do Diagrama de Classes e de Sequência – Caso de Uso “Gerenciar Turmas”

O caso de uso “Gerenciar Turmas” O coordenador pode criar, editar ou excluir turmas, atribuir professores e alunos a cada uma delas, e supervisionar a organização acadêmica do sistema.

1. Diagrama de Classes – Gerenciar Turmas

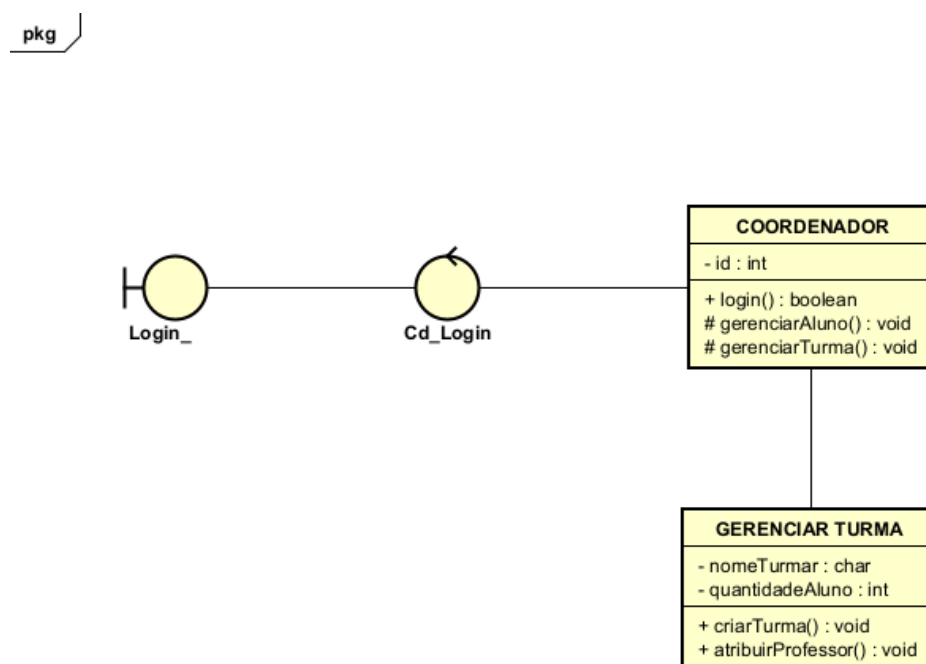
O diagrama de classes apresenta a estrutura estática relacionada à funcionalidade de gerenciamento de turmas. Ele destaca as classes Coordenador e Gerenciar Turma, além dos componentes de controle de login necessários para a autenticação do usuário.

- A classe Coordenador contém os atributos e métodos necessários para o controle administrativo, como *id*, *login()*, *gerenciarAluno()* e *gerenciarTurma()*.

- A classe Gerenciar Turma representa o módulo responsável por armazenar e manipular informações sobre as turmas. Ela possui atributos como *nomeTurma* e *quantidadeAluno*, e métodos como *criarTurma()* e *atribuirProfessor()*, que permitem a criação e vinculação das turmas aos professores.
- O relacionamento entre as classes demonstra que o coordenador depende do processo de autenticação (*Login_* e *Cd_Login*) para acessar as operações administrativas. Assim, o sistema garante segurança e controle sobre quem pode manipular os dados das turmas.

Esse diagrama ilustra a estrutura lógica da funcionalidade, destacando a organização dos objetos e as relações entre eles para assegurar uma implementação clara, modular e segura dentro da arquitetura do sistema acadêmico.

Figura 20: Diagrama de Classes – Gerenciar Turma.



2. Diagrama de Sequência – Gerenciar Turmas

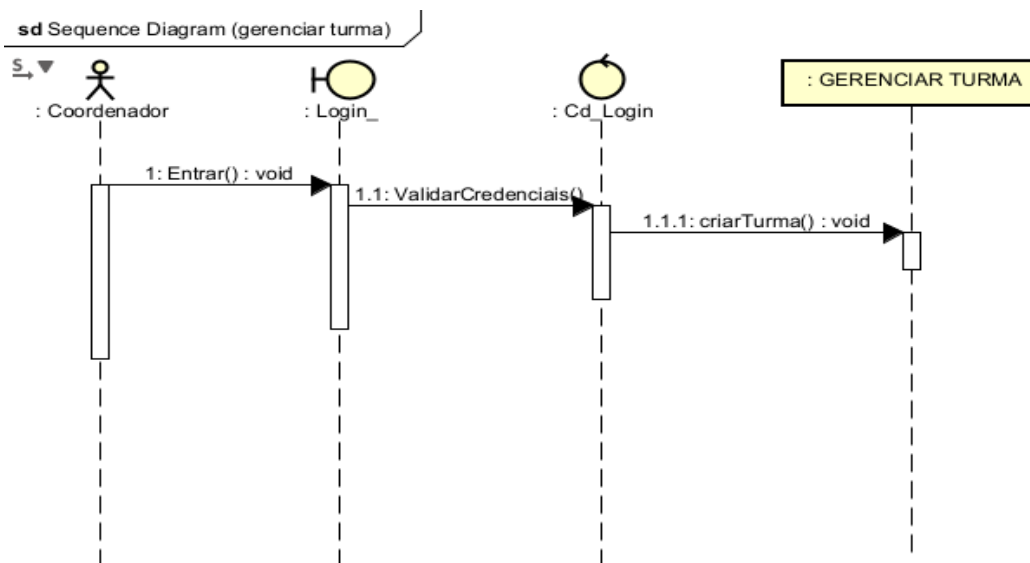
O diagrama de sequência descreve o fluxo dinâmico da interação entre os objetos durante a execução do caso de uso “Gerenciar Turmas”.

1. O processo se inicia quando o Coordenador executa a ação *Entrar()* para acessar o sistema.
2. Em seguida, o objeto *Login_* é acionado e envia uma mensagem para o controlador *Cd_Login*, responsável por realizar a operação *ValidarCredenciais()*.
3. Após a autenticação bem-sucedida, o controle é transferido para o módulo *Gerenciar Turma*, que executa o método *criarTurma()* — representando o momento em que o coordenador cria uma nova turma no sistema.

Esse diagrama evidencia o comportamento temporal dos objetos e o encadeamento das mensagens entre eles. Cada interação segue a lógica de autenticação e autorização antes de permitir qualquer modificação de dados, reforçando a importância do controle de acesso e da segurança no sistema.

Assim, o diagrama demonstra de forma clara como os objetos colaboram para realizar a ação de gerenciamento, seguindo os princípios da Engenharia de Software Orientada a Objetos e a metodologia UML, contribuindo para a documentação técnica e o entendimento do funcionamento interno da aplicação.

Figura 21: Diagrama de Sequência – Gerenciar Turmas



Fonte: O grupo, 2025

3. Descrição do Diagrama de Classes e de Sequência – Caso de Uso “Acesso à Atividade”

O caso de uso “**Acesso à Atividade**” Permite que o aluno visualize as atividades e tarefas disponibilizadas pelos professores, podendo verificar prazos, descrições e status de envio

1. Diagrama de Classes – Acesso à Atividade

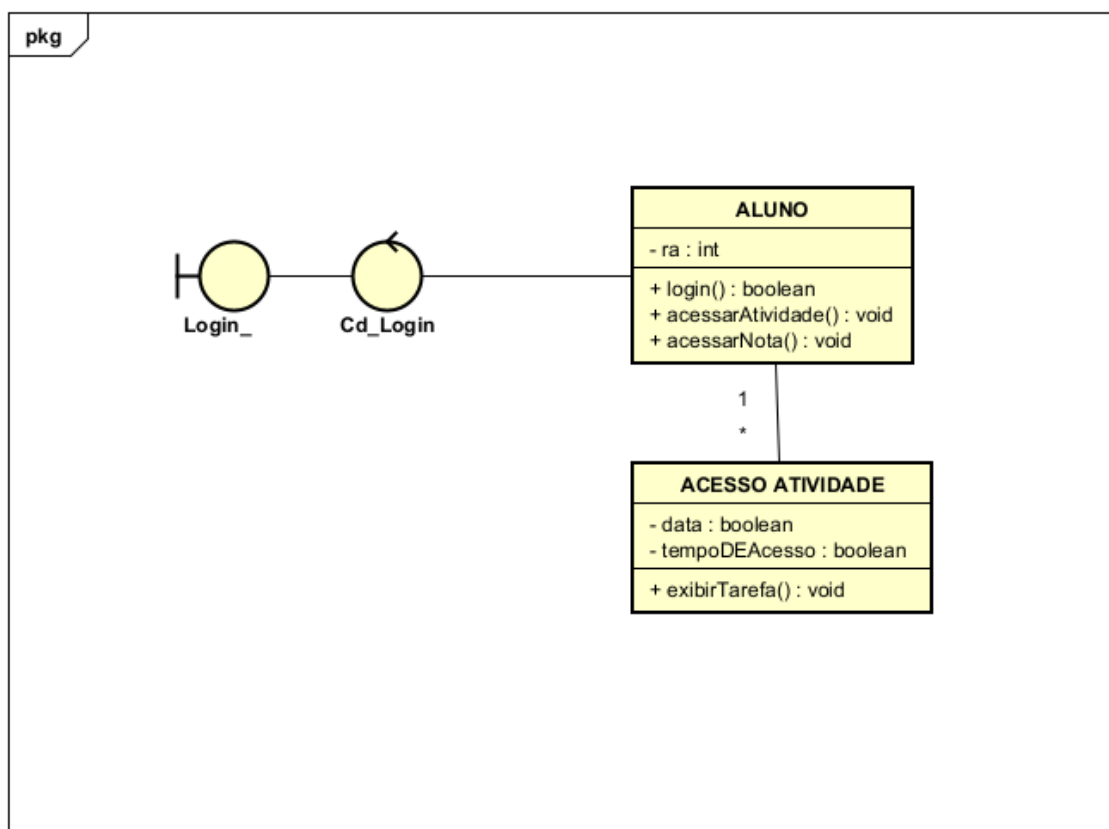
O diagrama de classes demonstra a estrutura estática do sistema para essa funcionalidade. Ele apresenta as classes Aluno e Acesso Atividade, além dos componentes responsáveis pelo processo de login e validação de credenciais (*Login_* e *Cd_Login*).

- A classe Aluno contém o atributo *ra* e métodos relacionados à autenticação e navegação no sistema, como *login()*, *acessarAtividade()* e *acessarNota()*. Ela representa o usuário principal deste caso de uso.
- A classe Acesso Atividade é responsável por armazenar e controlar as informações referentes às atividades acessadas, com atributos como *data* e *tempoDeAcesso*, e o método *exibirTarefa()*, que permite ao aluno visualizar a atividade desejada.

- O relacionamento entre as classes mostra que um aluno pode acessar várias atividades, caracterizado pela multiplicidade (1 para *Aluno* e *n* para *Acesso Atividade*).
- As classes *Login_* e *Cd_Login* fazem parte da camada de controle, sendo responsáveis pela autenticação do usuário e pela verificação de permissões antes de liberar o acesso ao conteúdo.

Esse diagrama ilustra como as classes colaboram entre si de forma organizada, reforçando a segurança e a integridade do sistema no acesso às atividades.

Figura 21: Diagrama de Classes – Acesso à Atividade



Fonte: O grupo, 2025. 3

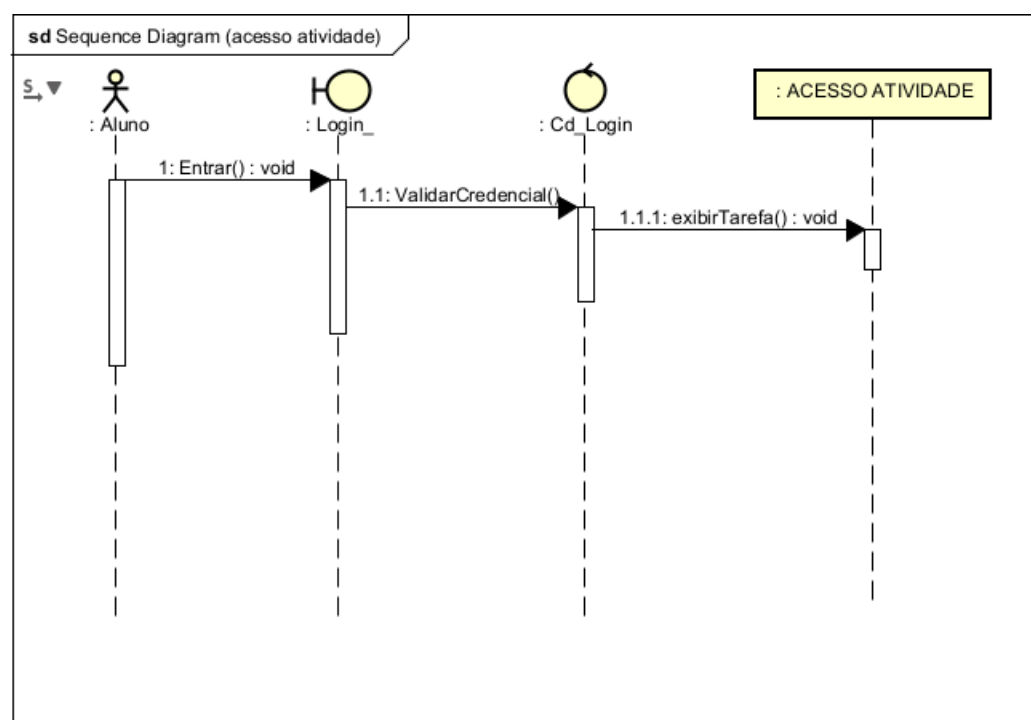
2. Diagrama de Sequência – Acesso à Atividade

O diagrama de sequência descreve o fluxo dinâmico de execução do caso de uso “Acesso à Atividade”, evidenciando a troca de mensagens entre os objetos do sistema ao longo do tempo.

1. O processo inicia quando o Aluno executa a ação *Entrar()* para acessar o sistema.
2. O objeto *Login_* recebe a solicitação e encaminha ao controlador *Cd_Login* a mensagem *ValidarCredencial()*, responsável por confirmar os dados de login.
3. Após a validação das credenciais, o sistema direciona o acesso ao módulo *Acesso Atividade*, que executa a operação *exibirTarefa()*, permitindo que o aluno visualize as atividades disponíveis.

Esse diagrama reflete a sequência lógica de autenticação e acesso dentro do sistema acadêmico, garantindo que apenas usuários autorizados possam visualizar as atividades. Ele demonstra o comportamento do sistema de forma clara e temporal, seguindo os princípios da Engenharia de Software Orientada a Objetos e da metodologia UML, que auxiliam na documentação e compreensão do funcionamento interno da aplicação.

Figura 22: Diagrama de Sequência – Acesso à Atividade



Fonte: O grupo, 2025.

3. Descrição do Diagrama de Classes e de Sequência – Caso de Uso “Editar Nota”

O caso de uso “Editar Nota” O professor lança, alterar ou corrigir as notas dos alunos em cada atividade, garantindo o controle e atualização do desempenho acadêmico.

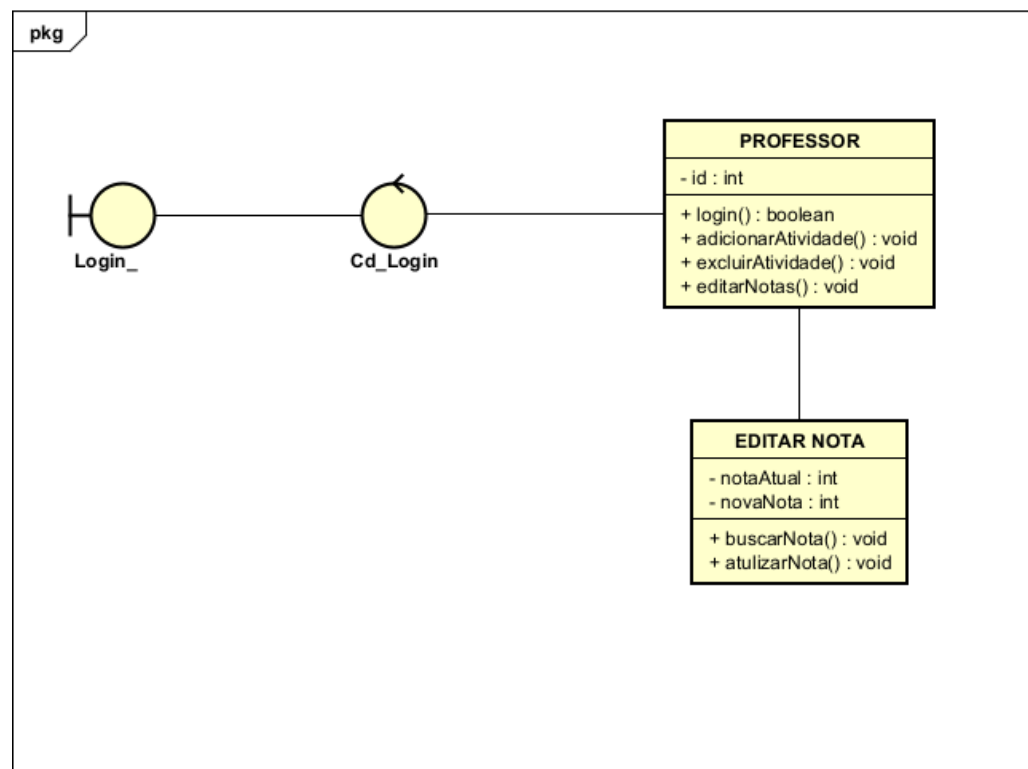
1. Diagrama de Classes – Editar Nota

O diagrama de classes demonstra a estrutura estática do sistema para a funcionalidade de edição de notas. Ele mostra como as classes estão relacionadas e quais métodos e atributos são utilizados nesse processo.

- A classe Professor contém o atributo *id* e métodos associados à gestão das atividades acadêmicas, como *adicionarAtividade()*, *excluirAtividade()* e *editarNotas()*. É o ator responsável por iniciar o processo de alteração de notas no sistema.
- A classe Editar Nota representa o módulo responsável pelo gerenciamento das informações de avaliação. Possui os atributos *notaAtual* e *novaNota*, que armazenam os valores antes e depois da atualização. Seus métodos *buscarNota()* e *atualizarNota()* executam, respectivamente, a busca pela nota existente e a substituição pelo novo valor inserido.
- As classes *Login_* e *Cd_Login* fazem parte da camada de controle, assegurando que o professor esteja devidamente autenticado antes de realizar qualquer modificação nas notas.

Esse modelo reforça o uso de herança e encapsulamento, princípios da programação orientada a objetos, garantindo segurança no acesso e clareza na estrutura do sistema.

Figura 23: Descrição do Diagrama de Classes e de Sequência – Caso de Uso “Editar Nota”



Fonte: O grupo, 2025.

2. Diagrama de Sequência – Editar Nota

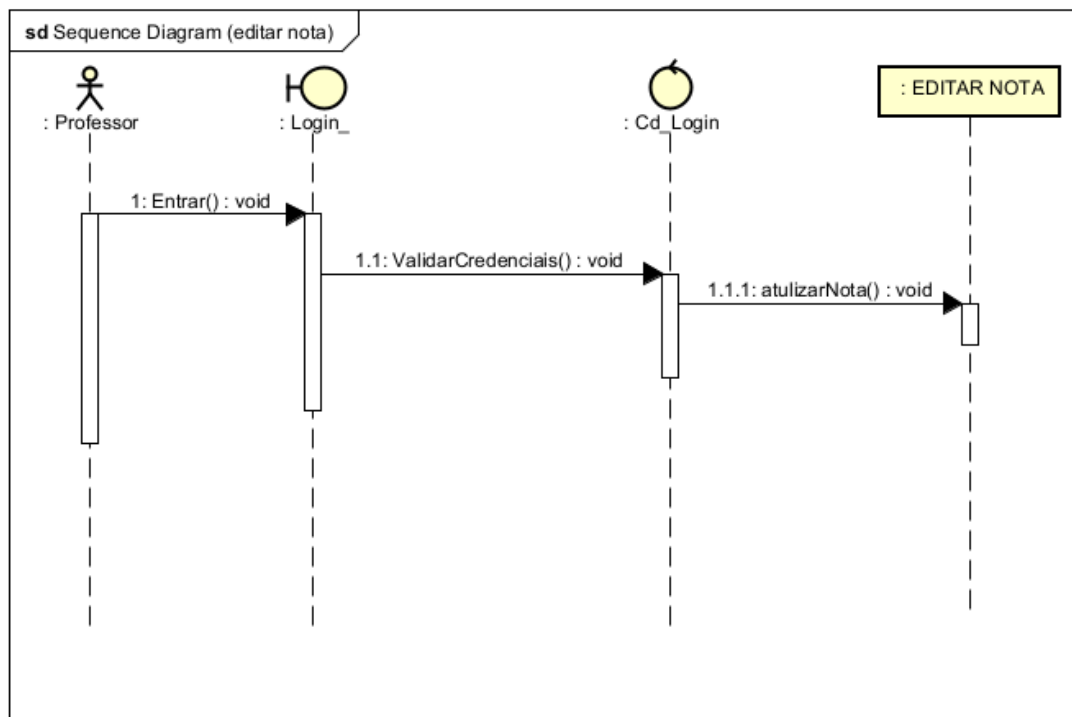
O diagrama de sequência descreve o comportamento dinâmico do sistema durante a execução do caso de uso “Editar Nota”. Ele apresenta a troca de mensagens entre os objetos envolvidos, evidenciando a ordem e o fluxo de execução das ações.

1. O processo inicia quando o Professor realiza a ação *Entrar()* para acessar o sistema.
2. O objeto *Login_* recebe a solicitação e envia uma mensagem para *Cd_Login*, responsável por executar o método *ValidarCredenciais()*, confirmando que o usuário possui permissão para realizar alterações.
3. Após a autenticação, o controle é passado ao módulo *Editar Nota*, que executa a operação *atualizarNota()*, registrando a nova nota do aluno no sistema.

Esse fluxo demonstra a importância da validação de credenciais e da sequência lógica de ações, assegurando que somente usuários autorizados possam modificar os dados acadêmicos. O diagrama também reforça a organização modular

do sistema, onde cada componente possui uma função específica e bem definida, seguindo os padrões da UML (Unified Modeling Language) e os princípios da Engenharia de Software Ágil aplicados no desenvolvimento do projeto.

Figura 24: Diagrama de Sequência – Editar Nota



Fonte: O grupo, 2025.

10.5.1 REDE DE COMPUTADORES E SISTEMA DISTRIBUIDOS

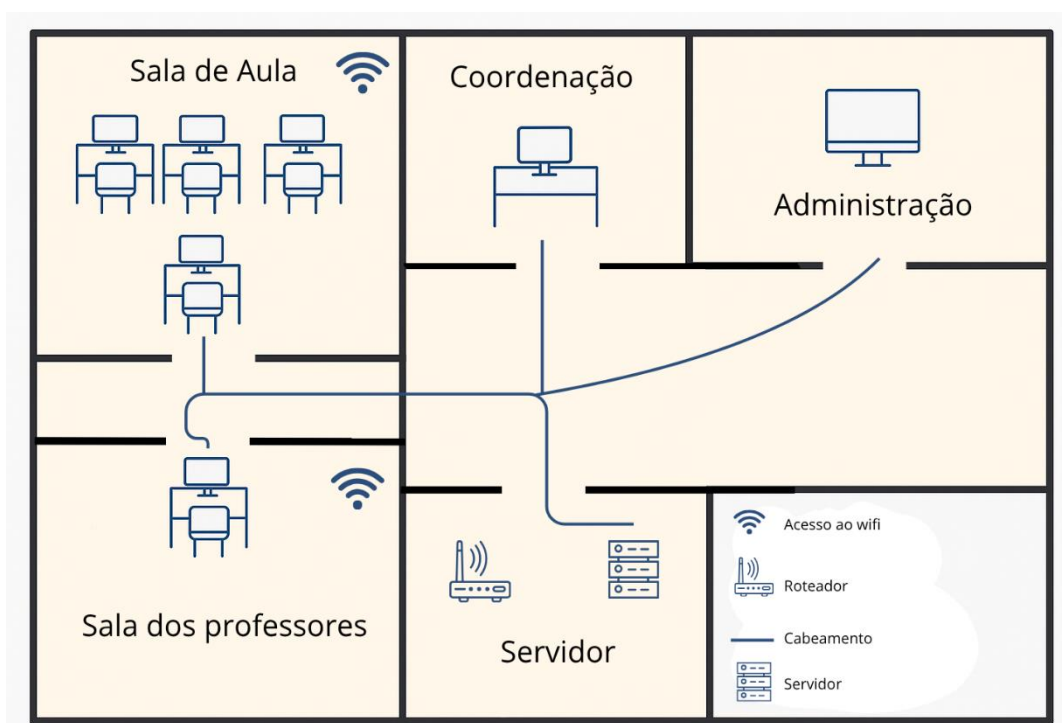
A planta baixa representa a distribuição física da rede de dados e comunicação em um ambiente educacional. O layout foi projetado para otimizar a cobertura de sinal e o desempenho das conexões.

- Sala de Aula: equipada com estações de trabalho conectadas via Wi-Fi, utilizando um ponto de acesso dedicado que distribui o sinal de forma uniforme.
- Sala dos Professores: segue o mesmo padrão, com um Access Point próprio para garantir conectividade estável.
- Coordenação e Administração: possuem computadores conectados por cabeamento estruturado, priorizando a segurança e estabilidade da conexão em setores administrativos.

- Sala de Servidores: abriga o roteador principal, responsável pela distribuição de sinal para toda a rede, e o servidor local, que hospeda os sistemas internos e controla os serviços de rede, como DHCP (atribuição automática de endereços IP) e DNS (resolução de nomes de domínio).

Os cabos de rede interligam os setores administrativos e pedagógicos diretamente ao servidor, reduzindo a latência e evitando sobrecarga na transmissão de dados. A rede sem fio é segmentada logicamente (VLAN) para separar o tráfego administrativo e o acadêmico, garantindo maior segurança e controle de acesso.

Figura 24:



Fonte: O grupo, 2025.

11. CONCLUSÃO

O desenvolvimento do Sistema Acadêmico Colaborativo com apoio de Inteligência Artificial permitiu comprovar que a aplicação integrada de diferentes linguagens e metodologias pode gerar soluções eficientes para o ambiente educacional. A pergunta-problema que orientou o projeto — como otimizar o gerenciamento acadêmico por meio de um sistema digital colaborativo e inteligente — foi respondida de forma positiva, uma vez que o sistema desenvolvido se mostrou capaz de centralizar informações, reduzir processos manuais e promover maior interação entre alunos, professores e coordenadores.

As hipóteses levantadas, relacionadas à possibilidade de automatizar tarefas, aprimorar a comunicação e adotar práticas sustentáveis, foram confirmadas ao longo do processo de implementação. A utilização das linguagens C e Python, combinada com metodologias ágeis e modelagem UML, possibilitou a criação de uma plataforma funcional e acessível, demonstrando a importância da integração entre teoria e prática no desenvolvimento de sistemas.

Constatou-se ainda que o uso da Inteligência Artificial agregou valor ao projeto, oferecendo recursos de análise e suporte à tomada de decisão. Assim, conclui-se que o projeto atingiu seus objetivos, contribuindo para o aprimoramento da gestão educacional e para a compreensão de como a tecnologia pode atuar como ferramenta estratégica no contexto acadêmico.

12. REFERÊNCIAS

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. **Lei Geral de Proteção de Dados Pessoais (LGPD)**. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm. Acesso em: 13 set. 2025.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum: o guia definitivo para o Scrum, as regras do jogo**. 2020. Disponível em: <https://scrumguides.org>. Acesso em: 13 set. 2025.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.
WINTERS, Titus; MANSHRECK, Tom; WRIGHT, Hyrum. **Software Engineering at Google: Lessons Learned from Programming Over Time**. Sebastopol: O'Reilly Media, 2020. Acesso em: 03 out.2025.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016. Acesso em: 07 out 2025.

HOLMES, Wayne; BIALIK, Maya; FADDA, Charles. **Artificial Intelligence in Education: Promises and Implications for Teaching and Learning**. Boston: Center for Curriculum Redesign, 2019. Acesso em: 10 out 2025.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH, 2021. Acesso em 12 de out 2025.

RUSSELL, Stuart J.; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3. ed. Upper Saddle River: Prentice Hall, 2010. Acesso em 12 de out 2025.

KUROSE, James F.; ROSS, Keith W. **Redes de Computadores e a Internet**. 8. ed. Pearson, 2021. Acesso em 12 de out 2025.

TANENBAUM, Andrew S.; WETHERALL, David J. **Redes de Computadores**. 5. ed. Pearson, 2019. Acesso em 17 de out 2025.

FOROUZAN, Behrouz A. **Comunicação de Dados e Redes de Computadores**. AMGH, 2018. Acesso em 18 de out 2025.

IEEE Standards Association. IEEE 802.3 & IEEE 802.11 **Standards for Networking**, 2024. Acesso em 18 de out 2025.

DIAS, Genebaldo Freire. **Educação Ambiental: princípios e práticas**. 12. ed. São Paulo: Gaia, 2017. Acesso em 22 de out 2025.

LEFF, Enrique. **Saber ambiental: sustentabilidade, racionalidade, complexidade e poder**. 9. ed. Petrópolis: Vozes, 2011. Acesso em 27 de out 2025.

PHILIPPI JR., Arlindo; PELICIONI, Maria Cecília Focesi. **Educação Ambiental e Sustentabilidade**. Barueri: Manole, 2005. Acesso em 31 de out 2025.

TIDD, Joe; BESSANT, John. **Gestão da inovação: integrando inovação, tecnologia e mudança organizacional**. 5. ed. Porto Alegre: Bookman, 2015. Acesso em 31 de out 2025.

DRUCKER, Peter Ferdinand. **Inovação e espírito empreendedor (entrepreneurship): prática e princípios**. São Paulo: Cengage Learning, 2014. Acesso em 02 de nov 2025.

OECD. Oslo Manual 2018: **Guidelines for Collecting, Reporting and Using Data on Innovation**. 4th ed. Paris: OECD Publishing, 2018. Acesso em 02 de nov 2025.

PHILIPPI JR., Arlindo; PELICIONI, Maria Cecília Focesi. **Educação Ambiental e Sustentabilidade**. Barueri: Manole, 2005. Acesso em 04 de nov 2025.

FICHA DE CONTROLE DO PIM

Grupo Nº _____ Ano: 2025 - Período: 2º - Orientador: Roberto Cordeiro Waltz.

Tema: Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA

Alunos:

RA	Nome	E-mail	Curso	Visto do aluno
H59DAH1	ARTHUR FERNANDES REIS	arthur.reis9@aluno.unip.br	ADS	
H658GF0	JOÃO EDUARDO SILVA NEIVA NEU	joao.neu@aluno.unip.br	ADS	
H70IFB0	JOÃO PEDRO AMÉRICO DA SILVA	joao.silva424@aluno.unip.br	ADS	
F3618A6	LUCAS MATHEUS MARQUES GONÇALVES	lucas.goncalves110@aluno.unip.br	ADS	
R873528	MIGUEL DINIZ BARRETO DOS SANTOS FILHO	miguel.santo1@aluno.unip.br	ADS	
H6619J8	VITOR DE MOURA DA SILVA	vitor.silva422@aluno.unip.br	ADS	

Registros:

Data do encontro	Observações
15.09.2025	Houve a definição da equipe e tratativas iniciais sobre o tema, divisão das disciplinas e das responsabilidades entre os integrantes do grupo.
25.09.2025	Nos reunimos para debater as ideias e assuntos que foram levantados, fomentando a ideia para execução.
10.10.2025	Desenvolvimento inicial do sistema e discussões sobre a execução do programa.
22.10.2025	Alguns membros da equipe apresentam a ideia para o professor responsável de cada disciplina, tiram dúvida e aproveitam a oportunidade para ajustar detalhes conforme instruções. Parte teórica já bem adiantada.
26.10.2025	Equipe se reúne após orientação do professor e verifica se os detalhes estão de acordo e corrigidos, encaminhando para finalização da parte de execução e focado na parte teórica.
08.11.2025	Equipe faz chamada com todos e o trabalho é exposto para percepção geral do grupo, apontamentos de melhoria são corrigidos imediatamente e novo encontro é marcado para 09.11.2025 afim de

	acabar com detalhes finais, deixando o projeto pronto para ser entregue
09.11.2025	Equipe faz a chamada com a presença de todos e alinham detalhes da entrega, combinam de imprimir e encadernar o projeto, responsável pela postagem em sistema reafirma a data.